The `--ini` switch isn't valid for all types of PHP binary (such as `php-fpm`), in which case we can run `php-fpm -i` for extremely detailed information dump:

```
$ php-fpm -i | grep "Configuration File"
Configuration File (php.ini) Path => /etc/php/8.4/fpm
Loaded Configuration File => /etc/php/8.4/fpm/php.ini
```

You might need to install it first, e.g., with `sudo apt-get install -y php8.4-fpm` or equivalent. Not all package managers bundle `php-fpm` with the rest of the core PHP packages, and not all php-fpm packages install a `php-fpm` CLI binary. If all else fails, use Docker:

```
$ docker run --rm php:8.4-fpm php-fpm -i | grep "Configuration File"
Configuration File (php.ini) Path => /usr/local/etc/php
...
```

Overriding `.ini` Settings

It is quite common for PHP to be used in multiple ways on the same system; the same codebase might target both web and CLI SAPIs. It is sometimes useful (or necessary) to adjust directives during execution or set (or override) values in the configuration file.

The `.ini` file syntax contains support for reading system environment variables, using the syntax `directive = ${ENV_VAR}`. Search your `php.ini` file with grep (the example below assumes a Debian-like config path; your system might differ):

```
$ grep 'memory_limit' /etc/php/8.4/cli/php.ini
memory_limit = ${PHP_MEMORY_LIMIT}

$ export PHP_MEMORY_LIMIT=128M
$ php -r "var_dump(ini_get('memory_limit'));"
string(4) "128M"
```

Here we have set the directive `memory_limit` to be `${PHP_MEMORY_LIMIT}`, then set the environment variable in bash with `export PHP_MEMORY_LIMIT=128M`. The final command is a single PHP statement `var_dump(ini_get('memory_limit'));` which returns the value. This way it is easy to configure PHP on a per-process basis, which is useful when you need to tune settings in the shell environment (every process in a Linux system has its own set of environment variables).